Benchmarking User-Specific Link Traversal Query Optimization: Evidence-Based Simulation of Decentralized Query Patterns

Ruben Eschauzier[1][0000−0002−6475−806X] and Ruben Taelman[1][0000−0001−5118−256X]

Department of Electronics and Information Systems, Ghent University – imec

Abstract. Optimizing queries in decentralized environments, where data sources are often unknown and dynamically discovered, remains a major challenge due to the lack of prior knowledge about data distribution and availability. User-specific client-side query engines can mitigate this limitation by exploiting patterns in previously issued queries. However, existing benchmarks fail to accurately reflect the performance of such approaches, as they typically rely on repetitive or template-based query structures that lack realistic variability. Moreover, publicly available query logs from real decentralized applications are scarce or absent in the literature. Consequently, the evaluation of client-side decentralized query optimization techniques remains incomplete and cannot be performed under representative conditions. To fill this gap, we introduce an evidence-based simulation of client query usage patterns. This simulation enables systematic evaluation and comparison of client-side optimization techniques under realistic and reproducible conditions. We demonstrate its use by applying it to a novel caching-based link traversal query processing approach. To assess the impact of realistic query sequences, we compare the results against a benchmark that is not sequence-based, highlighting how query sequence simulation influences client-side optimization performance. We find that [placeholder for results]. We conclude: [placeholder for conclusion]

Keywords: First keyword · Second keyword · Another keyword.

1 Introduction

Centralizing data can simplify query processing, but in many real-world situations, it conflicts with privacy requirements and fine-grained access controls. Structured decentralized ecosystems such as Solid [46] and Mastodon [43] encourage users to keep their own data in small, independent stores that follow shared data models. This approach strengthens user autonomy, yet it makes query execution more difficult because engines must identify and query many heterogeneous data sources, each with its own access policies.

Traditional federated querying approaches [49,47,30] support decentralization of sources, but typically assume a small (10-100) set of well-described and consistently available endpoints [16]. These assumptions fail once data is spread across thousands of small units that enforce local access-control policies. In addition, enforcing fine-grained access control policies [19] on large sources with expressive interfaces introduces notable overhead [40], making standard approaches unsuitable for highly decentralized environments.

Newer decentralized environments, such as Solid, often expose data as lightweight HTTP resources [56] instead of highly expressive query interfaces. Many of these resources are not known in advance and can only be discovered during query execution. Their simplicity lowers the cost of enforcing fine-grained controls. This setting requires a federation model that can operate over a very large number of small sources, many of which may be encountered at runtime, without relying on prior aggregation. Traditional federated approaches are insufficient in this setting due to the assumption of a small number of large endpoints.

Link Traversal-based Query Processing (LTQP) [27] fits these needs. It discovers documents incrementally by following URIs encountered during execution, starting from an initial set of seed documents provided by either the user or the query. Its link-driven, asynchronous traversal supports fine-grained permissions and avoids the requirement to know the entire data distribution beforehand.

However, the fact that LTQP discovers documents incrementally creates significant challenges for efficient execution. Using the traditional optimize-then-execute paradigm is difficult, as the query itself determines which data will be accessed. As a result, the optimizer has no prior knowledge of the relevant data until the query's traversal has already begun.

Some approaches optimize query execution without prior knowledge by using adaptive query processing [24] and zero-knowledge query planning [26]. These yield modest but inconsistent performance gains, depending on the decentralized environment and the heuristics used [53]. However, other aspects of zero-knowledge LTQP optimization, such as link prioritization [29], are unlikely to improve query result arrival times in emerging decentralized environments [18].

Consequently, other works explore using precomputed server-side information as prior knowledge or relevance indexes to efficiently execute LTQP queries [54,55,23]. While these approaches offer greater performance benefits, they require the server to maintain these indexes. This raises the cost of serving resources and complicates privacy and access control.

To avoid this server-side overhead, engines can derive prior knowledge directly on the client. In decentralized settings, LTQP is inherently client-side, with one engine instance serving a single user. As users issue sequences of potentially correlated queries, the engine can identify frequently accessed data and its characteristics. This enables engines to implement *personalized* query optimization algorithms that leverage these access patterns to acquire the necessary prior knowledge.

Benchmarking these approaches requires a benchmark that reflects the query sequences and correlations that LTQP engines may encounter when used by real clients over real data. Existing benchmarks such as WatDiv [1] and BSBM [10] mainly instantiate multiple versions of the same query template and run them sequentially, often with repetitions to reduce statistical noise. This paradigm is not sufficient for evaluating client-side query optimization, because it either overstates performance due to repeated executions of the same query or template, or understates it when these instantiations are randomly mixed to avoid such overestimation.

In caching literature, which can be viewed as a form of personalized query optimization, the main evaluation methods are simulated micro benchmarks [34,25] and macro benchmarks [58,21,25,14,41] that use query sequences. These simulated benchmarks are often created specifically for a single paper, leading to a fragmented landscape of benchmarking methodology. Whenever possible, real query sequences are used [28,32,59]. For centralized SPARQL query optimization, such query logs [9,42,11,50] are publicly available. However, for personalized query optimization in LTQP, real-world query logs from applications built on structured decentralized environments are not yet available. To avoid the fragmented evaluation practices seen in caching literature, we need a benchmark that can simulate such query sequences in a way that is representative of real-world usage.

In this paper, we address this gap in benchmarking literature by extending SolidBench [53], an existing LTQP benchmark, to generate query sequences. These sequences capture three query engine usage patterns identified in the literature on social network applications, which SolidBench simulates, as well as patterns observed in real-world SPARQL query logs. This extension allows researchers to evaluate personalized query optimization techniques against patterns they are likely to encounter when these methods are deployed in real client-facing applications.

To test our benchmark, we implement three caching strategies in the LTQP engine Comunica [51,53]. These strategies are intentionally simple, but they provide a clear view of the benchmark's performance characteristics and will demonstrate (insert here when results are in). **{TODO**: Add summary of main contributions to this section at the end if space permits**}**

2 Literature Review

2.1 Existing Benchmarking Approaches

For the evaluation of client-side benchmarking techniques that rely on preceding queries, we primarily investigate caching literature, as this is a well-studied problem relevant to our work.

In the literature, we find three distinct benchmarking approaches: simulated micro benchmarks, simulated end-to-end query benchmarks, and real-world query logs.

Simulated micro benchmarks attempt to isolate performance gains of the benchmarked approaches by fixing certain aspects of query execution. This can range from fixing the cached data size [25] to controlling the percentage of query-relevant data present in the cache [38]. Fixing aspects of query execution makes it easier to isolate and understand the effect of specific caching mechanisms. However, it also gives an incomplete picture, because we don't know how those fixed conditions relate to real workloads or how the methods perform when those assumptions don't hold.

Simulated end-to-end query benchmarks use generated query sequences to evaluate performance. They aim to measure cache hit rate and overall execution behavior under the assumption that the simulated sequences reflect realistic usage. In practice, the simulation is usually built by extending an existing benchmark so that it generates sequences instead of isolated queries. The sequence order is often determined by simple rules or by random selection.

A common approach is to use queries from existing benchmarks such as the Star Schema Benchmark [39] or the Berlin SPARQL Benchmark [10]. These queries are then randomly divided into sequences [25,38,21], with each sequence executed using a shared cache. This approach is straightforward but not realistic. Prior work shows that real workloads contain user behavior patterns and temporal locality, and these patterns can be exploited by caching methods. Random sequences fail to capture these properties.

More realistic is to adjust the query generators of existing benchmarks to include patterns in the query sequences. Some works [58,34] use a Pareto distribution to decide what entity is used to instantiate a query template. This follows from the observation that many human activities follow a Pareto distribution [6], with many queries relating to a small portion of the entities.

A third approach uses simulated query sequences that reflect the access patterns of specific applications. One example is a workload that models data scientists exploring different aspects of the same entity—such as an author in a scholarly network—through consecutive, thematically related queries [14]. Such simulations often include multiple query sessions, with new sessions beginning according to a probability parameter p. Another example is a simulated application scenario focused on retrieving information about vacation destinations, which is used to assess caching effectiveness [34].

Real-world query logs Whenever available, query logs from actual applications form a highly relevant benchmarking approach. However, obtaining these query logs can be challenging. For SPARQL query optimization, query logs for a variety of endpoints are available, such as DBpedia [45,50,9] and Wikidata [50]. These logs are extensively used in benchmarking SPARQL-based approaches [59,41,32].

The evaluation of personalized LTQP optimization would ideally use real-world query logs, but such logs do not exist. To avoid the fragmented evaluation practices of current simulated approaches, the next best option is a unified simulated benchmark. A shared benchmark would provide a common basis for comparison, improve reproducibility, and prevent each paper from recreating its own evaluation setup.

2.2 Real-world Query Usage Patterns

To inform the benchmark design on what patterns should be simulated, we will outline three relevant client query usage patterns observed in the literature. The relevance is determined for our social network application use case.

User Interest-Based Query Behavior Work on social networks shows that users tend to form communities based on shared interests [36,15,7,31]. These studies also show that user–user and user–content graphs exhibit power-law degree distributions, small-world structure, and scale-free properties. The result is a dense core of high-degree nodes that connect many smaller, strongly clustered groups of low-degree nodes [36]. Simulation studies of social behavior further support the idea that interest-driven group formation is stable and can be modeled reliably [2,22].

Research shows that users interact more frequently with others who share similar interests [37]. As noted in the study:

> This confirms in a more formal way that shared interests between users are reflected in a higher degree of interaction between them—in an implicit or explicit manner.

These findings support the simulation of user-relatedness and interest-driven behavior when generating query sequences.

Logical Query Sessions Analysis of web browsing behavior shows that activity can be grouped into sessions [35,44,48]. Rather than being defined purely by time, these sessions are better represented as logical sessions [35,48], which capture activity focused on a specific topic or goal.

Logical sessions are constructed by initializing a set of sessions T and a mapping $N : U \to T$ from URLs to sessions. The procedure is as follows:

- If the referrer r is empty, create a new session t with root node u and set $N(u) = t$.
- If $N(r)$ is already defined, attach u to the session $N(r)$ as needed and set $N(u) = N(r)$.
- Otherwise, create a new session t with root node r and leaf node u, then set $N(r) = N(u) = t$.

This formulation ensures that sessions capture sequences of activity focused on a topic, rather than being defined solely by proximity in time. Logical sessions often involve backtracking. New sessions start in approximately 15% of clicks and follow a log-normal distribution. The average size and depth of logical sessions have log-normal means of 6.1 and 3, respectively.

These same patterns are also exhibited by queries generated during application navigation. Consider the use-case of a social media application built upon decentralized data stores. When users view their feed, they might interact by clicking the username of a post's creator, viewing the comments, or liking the post. In each instance, the parameters of the newly generated query are derived directly from the results of the preceding 'feed' query. These queries are thus chained together in a structure that mirrors logical web sessions. Furthermore, users might search for specific posts, people, or topics, reflecting standard logical session-switching behavior.

Consequently, interactive application usage generates sequences of interdependent requests [57], where subsequent queries rely heavily on preceding results. This is reflected in Facebook's TAO data store, which is explicitly optimized for workloads where data access patterns are driven by the user's step-by-step traversal of the social graph within the application interface [13].

Query Refinement Analysis of SPARQL queries issued to the DBpedia endpoint reveals that users frequently submit related queries in streaks [12,60]. These streaks consist of query sequences that share an underlying intention but undergo iterative refinement to achieve the user's objective. Previous studies initially introduced the concept of SPARQL query sessions specifically to analyze robotic queries [60,12].

In this benchmark, we focus on organic query sequences [60], which users generate through interactions with software or browsers [33]. This organic interaction likely represents the primary method users will use to navigate structured decentralized environments. Furthermore, we deprioritize robotic queries because their sequences can already be simulated by traditional benchmark approaches.

For both organic and robotic queries, analysis of total usage and reformulations (removals and additions) of operators over all query logs shows that filter, union, and optional operators account for the vast majority of reformulated queries.

Previous work [60] defined robotic query patterns, which can be simulated by current benchmarks [53,10,1] using instantiations of query templates.

Recently, an analysis of Wikidata query logs demonstrated that query development is an iterative process of design, debugging, and maintenance [5]. Based on this empirical data, exploratory querying can be classified into six distinct categories [5]:

- **Result Refinement:** Users apply additional constraints, such as filters and selective triple patterns, to reduce returned records.
- **Result Generalization:** Users retrieve more entities by broadening parameters, such as adding superclasses or increasing result limits.
- **Result Extension:** Users append new triple patterns to fetch additional attributes for selected entities.
- **Query Refinement:** Users adjust complex language features to precisely align the query with their initial intent.
- **Bug Fixing:** Users correct structural or syntax errors, such as invalid URIs, within an initial query.
- **Shifting Query Intent:** Users alter core query elements, such as predicates or filters, to pivot searches toward related topics.

These findings confirm that real-world querying relies on dynamic, step-by-step modifications rather than the static execution of independent templates. Consequently, existing evaluation frameworks fail to capture how users actively interact with data systems. Evaluating system performance for these exploratory query sequences remains a critical research challenge, explicitly requiring new benchmarks to validate progress [5]. Therefore, modeling these empirical sequence patterns is essential to accurately assess client-side query optimization techniques.

3 Query Sequence Benchmark

In this section, we detail our simulation methodology and the resulting hyperparameters. We base our sequence generator on SolidBench [53], which simulates realistic decentralized social media applications within the Solid ecosystem. SolidBench utilizes the Social Network Benchmark (SNB) dataset [17,3] and applies the Linked Data Benchmark Council (LDBC) choke point methodology [4]. Its workload combines standard SNB *interactive* queries with custom *discover* templates, which explicitly target Link Traversal Query Processing (LTQP) choke points in decentralized environments.

To support sequence-based evaluation, we extend three core components of the SolidBench ecosystem:

- **Query Generator:** We modify the generator to produce correlated query sequences that reflect empirical real-world usage patterns.
- **Dataset Generator:** We enhance the data generation process to model interest-based similarities between simulated users.
- **Benchmark Runner:** We integrate sequence execution logic into the existing runner to facilitate straightforward, reproducible sequence-based experiments and allow easy generation of default configuration files.

3.1 Simulation Methodology

This subsection details how we integrate the empirically identified usage patterns into our simulated query sequences.

Logical Query Sessions To model logical sessions, we categorize queries into four primary topics: person-specific messages, forum information, person attributes, and message attributes. Because queries in decentralized environments often depend on preceding results, we map the output types of each template to valid subsequent templates. This creates a directed graph of query progression that governs the flow of a logical session.

Within a session, a template's *instantiation values* are typically derived from the previous query's results. To realistically simulate this "click-through" behavior, we execute centralized equivalents of the LDBC SNB queries using QLever [8]. We then randomly sample the returned result set to instantiate the subsequent query template. If a query yields no results, we fall back to our default, interest-based instantiation methodology (Section 3.1).

Finally, we simulate logical session switching. Drawing from web browsing behavior, we assign each sequence a session-switching probability sampled from a log-normal distribution with a mean of 15%. During sequence generation, a session switch triggers a random choice with equal probability: the engine either starts a new session (using interest-based instantiation) or resumes a previous one (deriving instantiation values from that session's last executed query). To prevent repetitive queries and maintain a uniform distribution of template instantiations, we explicitly prohibit backtracking within our model. Additionally, we maintain a balanced template distribution by dynamically scaling down each template's selection probability proportionally to its overall frequency.

User Interest Modeling Individuals interact more frequently with users sharing similar interests [37]. To simulate this behavior, we use the underlying LDBC dataset utilized by SolidBench [17], which inherently models realistic user preferences. The LDBC generator, Datagen, produces person profiles containing specific interests, educational backgrounds, and workplaces. It links users based on these demographic and interest overlaps, yielding a non-random network topology with a realistic degree distribution.

We use this interest-driven topology to generate template instantiation values for the initial query of each logical session. (As established, subsequent queries derive their values from preceding result sets). Specifically, we construct a binary feature vector v_k for each user k:

$$v_k = (x_{k,1}, \ldots, x_{k,n}, y_{k,1}, \ldots, y_{k,m}), \tag{1}$$

where $x_{k,j} = 1$ if user k holds stated interest j, and $y_{k,l} = 1$ if user k completed study l.

For each sequence, we randomly select a base person k and compute the cosine similarity between v_k and the feature vectors of all other users in the dataset.

We then select template instantiation values probabilistically based on these similarity scores by applying a softmax function with a temperature parameter T:

$$P(c_{i,k}) = \frac{e^{s_i/T}}{\sum_{j=1}^{N} e^{s_j/T}}, \tag{2}$$

where s_i represents the similarity score for candidate person i. To minimize computational overhead at large scale factors, we restrict this softmax operation to the top N candidate entities (ranked by similarity) rather than the entire population.

Finally, for query templates requiring non-person instantiation values (such as posts or activities), we compute selection probabilities based on the similarity scores of the users who created that content.

Refinement Patterns The benchmark simulates query refinement by randomly applying composable transformations to an instantiated query template. Following our literature review, we specifically focus on x types of transformations.

To ensure these transformations meaningfully alter query execution characteristics, rather than superficially appending triples that maintain a one-to-one relation with the result set, we apply a choke point-based design methodology [4]. Specifically, we define the following planning choke points for client-side query optimization in decentralized environments, building upon the original SolidBench choke points [53]:

1. **P.1:** Addition or removal of a `FILTER`. The system requires prior knowledge to adjust the query plan effectively and estimate filter selectivity.
2. **P.2:** Addition or removal of `UNION` or `OPTIONAL` clauses, which can alter whether a `FILTER` can be pushed down.
3. **P.3:** Addition or removal of selective triple patterns.
4. **P.4:** Addition or removal of non-selective triple patterns.

Beyond query planning, these refinements also impact the traversal process of decentralized querying approaches (such as LTQP). We identify the following traversal choke points:

1. **T.1:** Refinements expand or contract the reachable sub-web due to predicate additions.
2. **T.2:** Refinements shift the reachable sub-web by substituting the traversal starting point.
3. **T.3:** Refinements change the number of hops required to obtain relevant data (corresponding to SolidBench choke points L.1–L.6 [52]).
4. **T.4:** Refinements alter the usability of structural indexes, such as type indexes (corresponding to SolidBench choke point L.8 [52]).

The generator applies these refinements sequentially to a base SolidBench query template. By default, every refinement pattern includes a reciprocal “undo” operation. The supported refinement operators include:

- **`OPTIONAL` and `UNION` blocks:** Adding new blocks or inserting triples into existing ones (including specific `UNION` branches).
- **Basic Graph Patterns (BGPs):** Adding new triples to existing BGPs.
- **`FILTER` statements:** Appending new query constraints.
- **Value substitution:** Replacing template instantiation values to start traversal from a different seed document.

The generator also supports variable instantiation within refinements, allowing users to bind pattern variables to specific values randomly selected from candidates provided through configuration files. Users can define custom query refinement patterns using JSON files to augment the provided default configurations. These defaults, along with documentation and pattern descriptions, are available on GitHub.

Through this flexible generator, we explicitly simulate the empirical exploratory usage patterns [5] identified in Section 2.2. We map these empirical patterns directly to our composable query transformations and choke points:

- **Result Refinement:** We simulate this by adding or removing `FILTER` constraints and selective triple patterns in the BGP to adjust query selectivity (Choke points P.1, P.3).
- **Result Generalization:** We replicate this by adding new triples to existing BGPs (Choke points P.3, P.4), mirroring how users expand queried classes and fetch related entities.
- **Result Extension:** We simulate this by adding triple patterns to BGPs, `OPTIONAL` blocks, and `UNION` blocks (Choke points P.3, P.4, P.2).
- **Query Refinement:** We model this through the addition or removal of `OPTIONAL` and `UNION` blocks (Choke point P.2).

We exclude the **Bug Fixing** and **Shifting Query Intent** patterns, as these do not easily translate into composable programmatic rules that guarantee valid queries regardless of application order. Finally, because our refinement rules are fully composable, the generator naturally reflects the empirical observation that exploratory patterns are not mutually exclusive and frequently co-occur within a single session.

3.2 Hyperparameters

The sequence generation process introduces several new hyperparameters, which are detailed in Table 1. To guarantee reproducibility despite extensive random sampling, a single seeded pseudo-random number generator (PRNG) is used througout the entire pipeline.

Table 1. LogT-normal distribution parameters (μ, σ) control the queries per sequence, queries per logical session, and the probability of switching logical sessions. Refinement parameters define the likelihood (`patternProbability`) and bounds (`min/maxRefinementLength`) of generating a refinement sequence for a given query. Instantiation relies on a softmax `temperature` and a `maxLogits` cutoff for similarity-based entity selection.

Category	**Parameter**	**Value**
Distributions (μ, σ)	Sequence Length	3.0, 0.2
	Session Length	1.7, 0.5
	Transition Probability	-2.0, 0.25
Refinements	`patternProbability`	0.1
	`min/maxRefinementLength`	2, [3–5]
Instantiation	`temperature`	0.1
	`maxLogits`	5–10

4 Caching in Link Traversal-based Query Processing

To validate the resulting query sequences generated by the SolidBenchSequence (SBS) benchmark, we implement three caching approaches in the Comunica link traversal engine [51,53].

{TODO: Briefly discuss the three approaches, their trade offs and expected behaviors**}**

All implementations of the previously mentioned approaches follow the RFC-9111 [20] spec, however as the dataset is static, all cache hits can be used without revalidation and are in practice never considered stale. Thus, the benchmark's cache hit rates are likely overstated compared to a live scenario.

5 Experiment Setup

{TODO: Fill in numbers of generated data**}** To test our benchmark we generate a SolidBench dataset with default scale = 0.1, resulting in x triples over y pods. For the query sequence generation, we generate 6 sequences with in total 136 query instantiations. The hyperparameters used for the generation are outlined in Table **??**.

The purpose of our experiment is two-fold, one is to evaluate the characteristics of the simulated query sequences. Specifically, the impact our three benchmark design choices have on the correlations between queries. For caching-based algorithms, this correlation is primarily measured in the *hit rate* and *evictions*, thus to evaluate our benchmark we will investigate these metrics. The other is to evaluate the actual effectiveness of the implemented caching approaches in a single-threaded link traversal engine. While caching is not a new concept, the introduced caching approaches have not been rigorously tested in the single threaded execution environment of Comunica. Thus, we can immediately use our proposed benchmark to investigate the performance characteristics of our proposed caching.

5.1 Cache Hit Rate Experiment

To investigate the theoretical performance benefits of caching in link traversal, we additionally run experiments with synthetically set cache hit rates.

{TODO: Explain the experiment**}**

This experiment will not only provide new insights into the possible benefits of caching approaches as different levels of performance, it will also show that for a given average hitrate in a sequence the overhead of evictions and cache management costs, as this cost is excluded from the synthetic hitrate experiments. Thus, giving a more complete view of the performance landscape of such approaches and the realism of our proposed benchmark.

6 Results

6.1 Global Cache Performance

Figure 1 presents the global caching performance profiles for each approach at various sizes against the no-cache baseline. As expected, all caching methods outperform the baseline.

For the unindexed cache, small (350,000 triples) and medium (700,000 triples) sizes perform similarly. The large cache (1,400,000 triples) shows significant improvement, indicating it reaches the critical capacity needed to retain entries across the entire query sequence. However, the baseline ultimately *solves* more total queries. This indicates that cache management overhead causes some long-running queries to fail before the deadline.

The indexed cache results support this, showing an even higher proportion of failed queries. This aligns with the hypothesis that higher indexing costs slow down execution during frequent cache evictions. Yet, when the indexed cache succeeds, it provides a larger performance boost than the unindexed cache. Additionally, the indexed cache has a lower critical capacity threshold, as medium and large sizes perform similarly.

Finally, the indexed cache with cardinality estimation performs best. It significantly improves query speeds and solves *more* queries than the baseline. For this method, increasing the cache size does not improve performance; in fact, the large cache performs worse than the small and medium versions. This suggests cardinality estimation benefits from a smaller, less stale cache, because the estimation process evaluates all cache entries rather than just traversal-relevant ones.

6.2 Logical Sessions

To evaluate the impact of logical sessions on caching performance, we analyze the hit rate deviation (Δ) from the overall average when queries initiate a new session, switch to an existing one, or remain in the current session. Table 2 shows that new sessions severely degrade hit rates, existing session switches cause moderate degradation, and intra-session queries improve performance. These effects are strongest in small and medium caches.

Consequently, optimal client-side cache sizing depends heavily on user session-switching behavior. These findings highlight the need for eviction policies that better retain cross-session entries and validate our simulation design; ignoring session dynamics would obscure critical real-world performance shifts.

Future work should investigate whether query sequences share a frequently accessed core of cache entries. Protecting these entries from eviction could mitigate the performance penalties of session switching.

6.3 Refinement Patterns

To evaluate iterative query refinement simulation, we analyze cache behavior across refinement depths. Figure 1 shows that hit ratios increase progressively

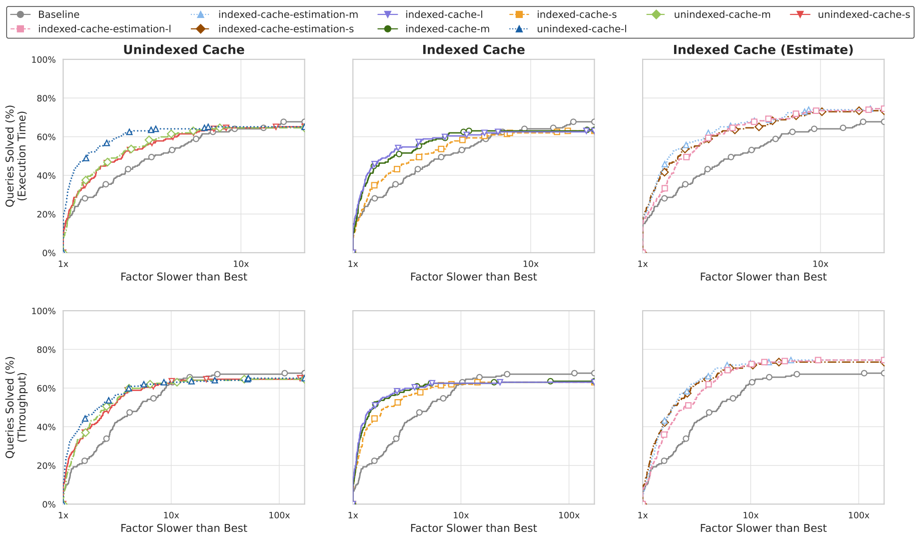

Fig. 1. **{TODO**: CAPTION HERE**}**

Table 2. Impact of logical session status on algorithm performance. Values represent absolute and percentage cache hit rate deviations from the global template average across new, existing, and within-session queries.

Cache Algorithm	**New Session**		**Existing Session**		**Within Session**	
	Abs.	**%**	**Abs.**	**%**	**Abs.**	**%**
unindexed-l	-0.066	-12.278	-0.014	-2.474	0.011	1.931
unindexed-m	-0.056	-15.649	-0.031	-7.900	0.024	5.486
unindexed-s	-0.071	-24.275	-0.021	-6.551	0.017	4.582
indexed-estimate-l	-0.053	-10.573	-0.015	-2.796	0.012	2.127
indexed-estimate-m	0.028	7.174	-0.000	-0.064	-0.000	-0.035
indexed-estimate-s	-0.044	-14.305	-0.013	-3.921	0.010	2.924
indexed-l	-0.026	-5.011	-0.003	-0.571	0.003	0.467
indexed-m	-0.087	-22.782	-0.021	-5.098	0.017	3.682
indexed-s	-0.033	-11.036	-0.015	-4.470	0.012	3.153

as queries undergo sequential refinement. Across all algorithms, hit rates increase at depths 2 and 3, confirming that refinement heavily reuses prior data.

At higher refinement depths, performance depends heavily on cache capacity. Large caches sustain high hit rates through depth 4, whereas small caches experience significant thrashing, and degraded hit ratios.

Surprisingly, the indexed cache without estimation maintains high hit rates even with smaller capacities. This counterintuitive result stems from implementation differences that influence how the single-threaded Node.js event loop schedules asynchronous traversal and join executions. This scheduling variance alters traversal order and rate, ultimately impacting cache eviction and hit performance.

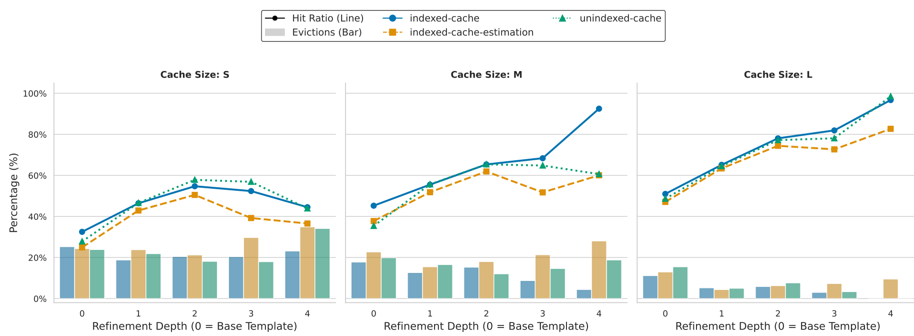

Fig. 2. Cache hit ratios and percentage of cache capacity evicted across refinement depths for small, medium, and large cache configurations.

The observed cache hitrate and eviction differences directly translate to measurable execution speedups. Figure 2 illustrates that the geometric mean speedup for mainly throughput generally scales positively with refinement depth. The indexed-cache algorithm proves particularly effective for prolonged exploratory sequences, achieving substantial throughput gains at depth 4 compared to the unindexed alternatives.

6.4 Theoretical Hit Rate Analysis

blabla we want to know what the scaling of speedup looks like when we arbitrarily simulate a cache hitrate for some algorithms. This does not take into account cost of eviction and caching.

From Figures 4 and 5, we find that in most cases caching can significantly improve query execution time, with improvements scaling predictably as hit rate increases. For some long-running queries such as interactive-discover-6, 7, and short-3 no speedup is observed, with some queries even showing slower result production. These query templates traverse many solid pods, but do not require all traversed data. In such cases, we hypothesize that having access to a

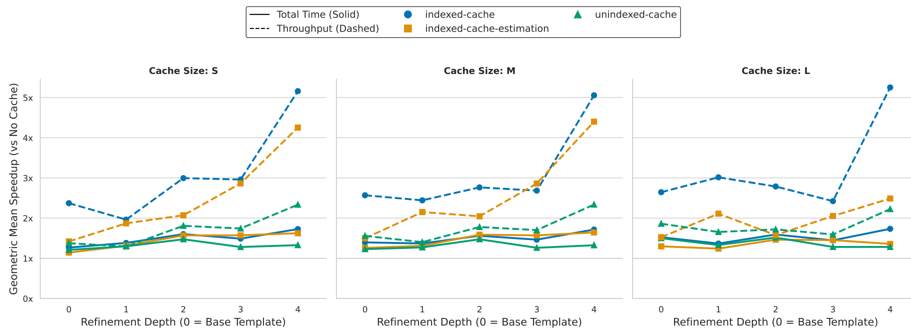

Fig. 3. Geometric mean of observed speedups and increases respectively for total execution time and query throughput relative to the no caching baseline across refinement depths.

cache which serves documents from memory will quickly produce too much data and slow down result production. Thus, waiting for HTTP responses between traversal steps allows the JavaScript event loop to run the join execution pipeline and thus produces results faster as only the first few requests are required for first results.

The two caching approaches exhibit similar overall performance, though the indexed cache significantly reduces the time-to-first-result across most templates. As expected, an indexed cache is highly advantageous when in the absence of evictions. However, the benchmark results demonstrate that the initial cost of indexing data introduces a clear trade-off: it amplifies both the potential speedups and slowdowns compared to an unindexed cache.

6.5 Benchmark abalations

References

1. Aluç, G., Hartig, O., Özsu, M.T., Daudjee, K.: Diversified stress testing of rdf data management systems. In: International Semantic Web Conference. pp. 197–212. Springer (2014)
2. Amblard, F., Bouadjio-Boulic, A., Gutiérrez, C.S., Gaudou, B.: Which models are used in social simulation to generate social networks? a review of 17 years of publications in jasss. In: 2015 Winter Simulation Conference (WSC). pp. 4021–4032. IEEE (2015)
3. Angles, R., Antal, J.B., Averbuch, A., Birler, A., Boncz, P., Búr, M., Erling, O., Gubichev, A., Haprian, V., Kaufmann, M., et al.: The ldbc social network benchmark (2020)
4. Angles, R., Boncz, P., Larriba-Pey, J., Fundulaki, I., Neumann, T., Erling, O., Neubauer, P., Martinez-Bazan, N., Kotsev, V., Toma, I.: The linked data benchmark council: a graph and rdf industry benchmarking effort. vol. 43, pp. 27–31. ACM New York, NY, USA (2014)

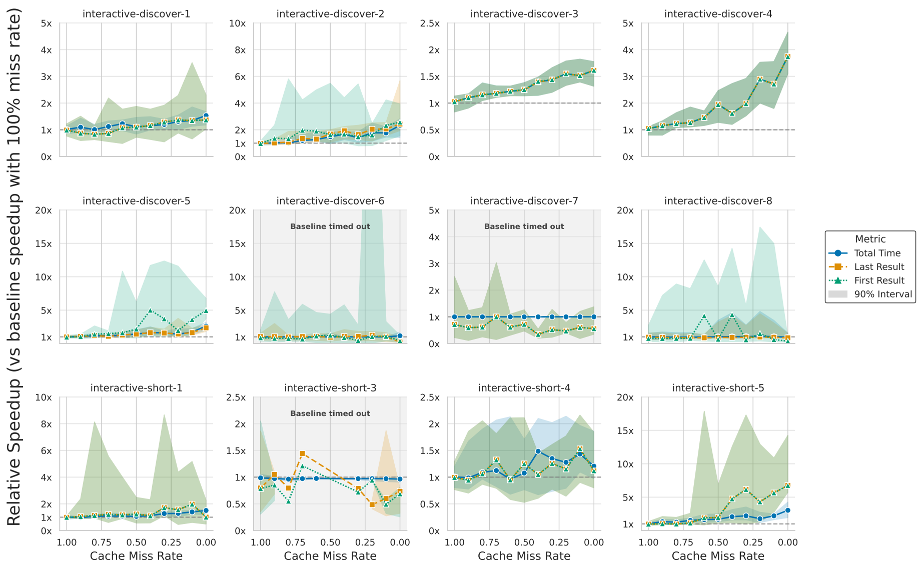

Fig. 4. Theoretical unindexed cache performance by query template. Lines indicate the mean relative speedup of the repeated query for total execution time, first result, and last result against a 100% miss rate baseline. Shaded regions denote the absolute minimum and maximum variance across iterations.

5. Arenas, M., Franconi, E., Hammerer, J., Hartig, O., Hose, K., Koesten, L., Konstantinidis, G., Libkin, L., Martens, W., Sasaki, Y., et al.: Exploring exploratory querying. Proceedings of the VLDB Endowment **18**(13), 5731–5739 (2025)
6. Arnold, B.C.: Pareto distribution. Wiley StatsRef: Statistics Reference Online pp. 1–10 (2014)
7. Backstrom, L., Huttenlocher, D., Kleinberg, J., Lan, X.: Group formation in large social networks: membership, growth, and evolution. In: Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining. pp. 44–54 (2006)
8. Bast, H., Buchhold, B.: Qlever: A query engine for efficient sparql+ text search. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management. pp. 647–656 (2017)
9. Berendt, B., Hollink, L., Luczak-Rösch, M., Möller, K., Vallet, D.: Usewod2013 3rd international workshop on usage analysis and the web of data. 10th ESWC-Semantics and Big Data, Montpellier, France (2013)
10. Bizer, C., Schultz, A.: The berlin sparql benchmark. In: Semantic Services, Interoperability and Web Applications: Emerging Concepts, pp. 81–103. IGI Global Scientific Publishing (2011)
11. Bonifati, A., Martens, W., Timm, T.: DARQL: Deep Analysis of SPARQL Queries. In: Companion Proceedings of the The Web Conference 2018. pp. 187–190. WWW '18, International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE (Apr 2018). https://doi.org/10.1145/3184558.3186975, https://dl.acm.org/doi/10.1145/3184558.3186975

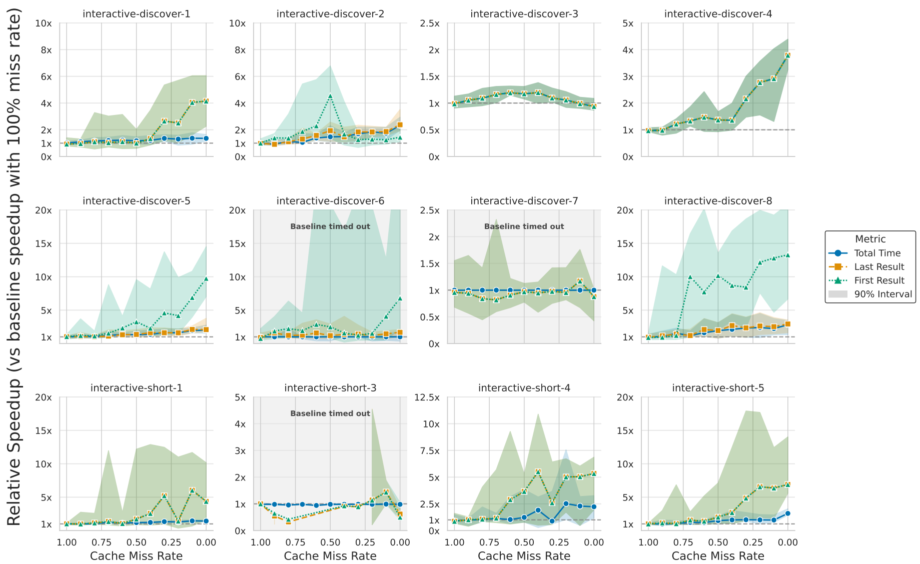

Fig. 5. Theoretical indexed cache performance by query template. Lines indicate the mean relative speedup of the repeated query for total execution time, first result, and last result against a 100% miss rate baseline. Shaded regions denote the absolute minimum and maximum variance across iterations.

12. Bonifati, A., Martens, W., Timm, T.: An analytical study of large sparql query logs. The VLDB Journal **29**(2), 655–679 (2020)
13. Bronson, N., Amsden, Z., Cabrera, G., Chakka, P., Dimov, P., Ding, H., Ferris, J., Giardullo, A., Kulkarni, S., Li, H., et al.: {TAO}:{Facebook's} distributed data store for the social graph. In: 2013 USENIX annual technical conference (USENIX ATC 13). pp. 49–60 (2013)
14. Chatzopoulos, S., Vergoulis, T., Skoutas, D., Dalamagas, T., Tryfonopoulos, C., Karras, P.: Atrapos: Evaluating metapath query workloads in real time. CoRR (2022)
15. Clark, A.: Understanding community: A review of networks, ties and contacts (2007)
16. Dang, M.H., Aimonier-Davat, J., Molli, P., Hartig, O., Skaf-Molli, H., Le Crom, Y.: Fedshop: A benchmark for testing the scalability of sparql federation engines. In: International Semantic Web Conference. pp. 285–301. Springer (2023)
17. Erling, O., Averbuch, A., Larriba-Pey, J., Chafi, H., Gubichev, A., Prat, A., Pham, M.D., Boncz, P.: The ldbc social network benchmark: Interactive workload. In: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data. pp. 619–630 (2015)
18. Eschauzier, R., Taelman, R., Verborgh, R.: Revisiting link prioritization for efficient traversal in structured decentralized environments. In: International Semantic Web Conference. pp. 575–593. Springer (2025)
19. Esteves, B., Pandit, H.J., Rodríguez-Doncel, V.: Odrl profile for expressing consent through granular access control policies in solid. In: 2021 IEEE European

Symposium on Security and Privacy Workshops (EuroS&PW). pp. 298–306. IEEE (2021)
20. Fielding, R., Nottingham, M., Reschke, J.: Rfc 9111: Http caching (2022)
21. Folz, P., Skaf-Molli, H., Molli, P.: Cyclades: a decentralized cache for triple pattern fragments. In: European semantic web conference. pp. 455–469. Springer (2016)
22. Hamill, L., Gilbert, G.: Social circles: A simple structure for agent-based social network models. Journal of Artificial Societies and Social Simulation **12**(2) (2009)
23. Hanski, J., Taelman, R., Verborgh, R.: Observations on bloom filters for traversal-based query execution over solid pods. In: European Semantic Web Conference. pp. 228–233. Springer (2024)
24. Hanski, J., Van Braeckel, S., Taelman, R., Verborgh, R.: Link traversal over decentralised environments using restart-based query planning. In: International Conference on Web Engineering (2025)
25. Hartig, O.: How caching improves efficiency and result completeness for querying linked data. In: LDOW (2011)
26. Hartig, O.: Zero-knowledge query planning for an iterator implementation of link traversal based query execution. In: Extended Semantic Web Conference. pp. 154–169. Springer (2011)
27. Hartig, O., Bizer, C., Freytag, J.C.: Executing sparql queries over the web of linked data. In: International semantic web conference. pp. 293–309. Springer (2009)
28. Hartig, O., Heath, T.: Populating personal linked data caches using context models. In: Proceedings of the 21st International Conference on World Wide Web. pp. 521–522 (2012)
29. Hartig, O., Özsu, M.T.: Walking without a map: Ranking-based traversal for querying linked data. In: International Semantic Web Conference. pp. 305–324. Springer (2016)
30. Heling, L., Acosta, M.: Federated sparql query processing over heterogeneous linked data fragments. In: Proceedings of the ACM Web Conference 2022. pp. 1047–1057 (2022)
31. Kalaitzakis, A., Papadakis, H., Fragopoulou, P.: Evolution of user activity and community formation in an online social network. In: Proceedings of the 2012 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining. pp. 1315–1320. IEEE (2012)
32. Lorey, J., Naumann, F.: Caching and prefetching strategies for sparql queries. In: Extended Semantic Web Conference. pp. 46–65. Springer (2013)
33. Malyshev, S., Krötzsch, M., González, L., Gonsior, J., Bielefeldt, A.: Getting the most out of wikidata: Semantic technology usage in wikipedia's knowledge graph. In: The Semantic Web–ISWC 2018: 17th International Semantic Web Conference, Monterey, CA, USA, October 8–12, 2018, Proceedings, Part II 17. pp. 376–394. Springer (2018)
34. Martin, M., Unbehauen, J., Auer, S.: Improving the performance of semantic web applications with sparql query caching. In: Extended Semantic Web Conference. pp. 304–318. Springer (2010)
35. Meiss, M., Duncan, J., Gonçalves, B., Ramasco, J.J., Menczer, F.: What's in a session: tracking individual behavior on the web. In: Proceedings of the 20th ACM conference on Hypertext and hypermedia. pp. 173–182. ACM, Torino Italy (Jun 2009). https://doi.org/10.1145/1557914.1557946, https://dl.acm.org/doi/10.1145/1557914.1557946
36. Mislove, A., Marcon, M., Gummadi, K.P., Druschel, P., Bhattacharjee, B.: Measurement and analysis of online social networks. In: Proceedings of the 7th ACM SIGCOMM conference on Internet measurement. pp. 29–42 (2007)

37. Mitzlaff, F., Atzmueller, M., Benz, D., Hotho, A., Stumme, G.: User-relatedness and community structure in social interaction networks. arXiv preprint arXiv:1309.3888 (2013)
38. Nicholson, H., Chrysogelos, P., Ailamaki, A.: Hpcache: memory-efficient olap through proportional caching revisited. The VLDB Journal **33**(6), 1775–1791 (2024)
39. O'Neil, P., O'Neil, E., Chen, X., Revilak, S.: The star schema benchmark and augmented fact table indexing. In: Technology Conference on Performance Evaluation and Benchmarking. pp. 237–252. Springer (2009)
40. Padia, A., Finin, T., Joshi, A., et al.: Attribute-based fine grained access control for triple stores. In: 3rd Society, Privacy and the Semantic Web-Policy and Technology workshop, 14th International Semantic Web Conference (2015)
41. Papailiou, N., Tsoumakos, D., Karras, P., Koziris, N.: Graph-aware, workload-adaptive sparql query caching. In: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data. pp. 1777–1792 (2015)
42. Picalausa, F., Vansummeren, S.: What are real SPARQL queries like? In: Proceedings of the International Workshop on Semantic Web Information Management. pp. 1–6. SWIM '11, Association for Computing Machinery, New York, NY, USA (Jun 2011). https://doi.org/10.1145/1999299.1999306, https://dl.acm.org/doi/10.1145/1999299.1999306
43. Raman, A., Joglekar, S., Cristofaro, E.D., Sastry, N., Tyson, G.: Challenges in the decentralised web: The mastodon case. In: Proceedings of the internet measurement conference. pp. 217–229 (2019)
44. Sadagopan, N., Li, J.: Characterizing typical and atypical user sessions in clickstreams. In: Proceedings of the 17th international conference on World Wide Web. pp. 885–894 (2008)
45. Saleem, M., Ali, M.I., Hogan, A., Mehmood, Q., Ngomo, A.C.N.: Lsq: the linked sparql queries dataset. In: International semantic web conference. pp. 261–269. Springer (2015)
46. Sambra, A.V., Mansour, E., Hawke, S., Zereba, M., Greco, N., Ghanem, A., Zagidulin, D., Aboulnaga, A., Berners-Lee, T.: Solid: a platform for decentralized social applications based on linked data. MIT CSAIL & Qatar Computing Research Institute, Tech. Rep. **2016** (2016)
47. Schmidt, M., Görlitz, O., Haase, P., Ladwig, G., Schwarte, A., Tran, T.: Fedbench: A benchmark suite for federated semantic data query processing. In: The Semantic Web–ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23-27, 2011, Proceedings, Part I 10. pp. 585–600. Springer (2011)
48. Schneider, F., Feldmann, A., Krishnamurthy, B., Willinger, W.: Understanding online social network usage from a network perspective. In: Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement. pp. 35–48 (2009)
49. Schwarte, A., Haase, P., Hose, K., Schenkel, R., Schmidt, M.: Fedx: Optimization techniques for federated query processing on linked data. In: The Semantic Web–ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23-27, 2011, Proceedings, Part I 10. pp. 601–616. Springer (2011)
50. Stadler, C., Saleem, M., Mehmood, Q., Buil-Aranda, C., Dumontier, M., Hogan, A., Ngonga Ngomo, A.C.: Lsq 2.0: A linked dataset of sparql query logs. Semantic Web **15**(1), 167–189 (2024)
51. Taelman, R., Van Herwegen, J., Vander Sande, M., Verborgh, R.: Comunica: a modular sparql query engine for the web. In: International Semantic Web Conference. pp. 239–255. Springer (2018)

52. Taelman, R., Verborgh, R.: Evaluation of link traversal query execution over decentralized environments with structural assumptions (2023)
53. Taelman, R., Verborgh, R.: Link traversal query processing over decentralized environments with structural assumptions. In: International Semantic Web Conference. pp. 3–22. Springer (2023)
54. Tam, B.E., Taelman, R., Colpaert, P., Verborgh, R.: Opportunities for shape-based optimization of link traversal queries. arXiv preprint arXiv:2407.00998 (2024)
55. Umbrich, J., Hose, K., Karnstedt, M., Harth, A., Polleres, A.: Comparing data summaries for processing live queries over linked data. World Wide Web **14**(5), 495–544 (2011)
56. Verborgh, R., Vander Sande, M., Hartig, O., Van Herwegen, J., De Vocht, L., De Meester, B., Haesendonck, G., Colpaert, P.: Triple pattern fragments: a low-cost knowledge graph interface for the web. Journal of Web Semantics **37**, 184–206 (2016)
57. Vögele, C., van Hoorn, A., Schulz, E., Hasselbring, W., Krcmar, H.: Wessbas: extraction of probabilistic workload specifications for load testing and performance prediction—a model-driven approach for session-based application systems. Software & Systems Modeling **17**(2), 443–477 (2018)
58. Williams, G.T., Weaver, J.: Enabling fine-grained http caching of sparql query results. In: International Semantic Web Conference. pp. 762–777. Springer (2011)
59. Zhang, W.E., Sheng, Q.Z., Qin, Y., Yao, L., Shemshadi, A., Taylor, K.: Secf: improving sparql querying performance with proactive fetching and caching. In: Proceedings of the 31st Annual ACM Symposium on Applied Computing. pp. 362–367 (2016)
60. Zhang, X., Wang, M., Zhao, B., Liu, R., Zhang, J., Yang, H.: Characterizing robotic and organic query in sparql search sessions. In: Web and Big Data: 4th International Joint Conference, APWeb-WAIM 2020, Tianjin, China, September 18-20, 2020, Proceedings, Part I 4. pp. 270–285. Springer (2020)